# Predicting Pokémon Battles by Rebuilding What the Log Hides

Matteo Angeloro  Lorenzo Amoruso  Carmine [TODO: surname]
Lorenzo Valenti
Sapienza University of Rome
[TODO: institutional emails]

## Abstract

*We predict whether Player 1 wins a Gen 2 OU Pokémon battle from its first 30 recorded turns. The key observation is that the battle log hides most of Player 2's knockouts: a fainted Player 2 Pokémon looks like an ordinary switch. Using Player 1's always-visible knockouts as a control group, a classifier estimates which Player 2 switches were knockouts, and the corrected game state feeds a LightGBM model together with an out-of-fold encoding of Player 1's team history. The model reaches 0.699 accuracy in repeated cross-validation, 0.700 on the public leaderboard and 0.683 on the private one.*

## 1. Introduction

The challenge gives 10,000 labelled and 5,000 test battles. For each battle we see Player 1's full team, Player 2's lead and the first 30 recorded turns (active Pokémon, HP, status, boosts, moves); the target is whether Player 1 wins, and the metric is accuracy. Battles last 30 to 80 turns, so the outcome is still open when the log ends.

Two facts shaped our solution. First, Player 2's `fnt` state appears almost only in double knockouts or at the very last record: about one Player 2 knockout per battle is hidden. Second, the same Player 1 team recurs across battles, and its win rate elsewhere predicts the outcome, while Player 2's does not.

## 2. Evaluated Approaches

**Feature engineering.** A first model on the state at turn 30 taken at face value (Pokémon alive and their HP, statuses, active Pokémon, boosts, Spikes) reached 0.675. Adding Player 1's species and items and the difference of move counts between the players gave a baseline of 0.685. We then added two blocks. *Hidden knockouts*: for every Pokémon that leaves and never returns (a "vanishing"), a LightGBM classifier separates Player 2's events from Player 1's using only the event description (HP before leaving, last HP drop, whether the replacement entered unhurt, move powers, the opponent's HP drop). Since Player 1's vanishings are all real switches, the density ratio gives $P(\mathrm{KO} \mid x) = 1 - c(1-q)/q$, with $q$ the classifier output and $c$ fixing the total to the observed excess. No labels are used; with these estimates Player 2 loses as many Pokémon as Player 1 (1.61 vs. 1.69 per battle). The state features are then recomputed weighting each Pokémon by its probability of still being alive. *Player 1's history*: the smoothed win rate of Player 1's exact team, as a target encoding computed out-of-fold inside each training fold. We kept a block only if it improved the paired CV accuracy. Player 2's bench estimated from the team pool, extra state features (sleep counters, matchups, HP trajectories) and a damage formula were within noise and dropped.

**Models.** LightGBM with shallow trees was our final learner. Logistic regression was 0.5 points worse; XGBoost, CatBoost, seed bagging and blends all stayed within $\pm 0.003$ of LightGBM.

**Validation.** Every configuration was scored on the same stratified 5-fold CV repeated three times, and configurations were compared through paired fold differences with the Nadeau–Bengio corrected standard error [1]. Choices were confirmed on a second seed.

## 3. Results

Table 1. CV accuracy (5-fold $\times$ 3) and paired gain over the baseline ($\pm$ corrected SE).

| Configuration | CV acc. | Gain |
|---|---|---|
| State at face value | 0.675 | $-0.010$ |
| Baseline (+ species, items, moves) | 0.685 | – |
| Final without hidden knockouts | 0.687 | $+0.002$ |
| Final without P1 history | 0.697 | $+0.012$ |
| **Final model** | **0.699** | $\mathbf{+0.014 \pm 0.004}$ |

Table 1 summarises the ablation. Almost all of the gain comes from the hidden-knockout correction ($+0.012 \pm 0.003$); the history adds $+0.002 \pm 0.004$ once species are present. It scored 0.700 on the public and 0.683 on the private leaderboard. The private score is 1.5 points below CV (two standard errors of a 3,500-battle split), and our best submissions lie within noise of each other there. Code: https://github.com/teoang/fds-pokemon-challenge.

## References

[1] Claude Nadeau and Yoshua Bengio. Inference for the generalization error. *Machine Learning*, 52(3):239–281, 2003. 1